

[CS 153] Web 基础设施的扩展与适应 — Vercel CEO Guillermo Rauch

基于 Stanford CS 153 课程内容整理

2025

视频信息

频道	Stanford CS 153
发布日期	2025-03-10
时长	37:16
链接	https://www.youtube.com/watch?v=9SqYFxp9yRM

目录

1 引言：为何在 AWS 之上再建一层？	3
1.1 从 Kubernetes 到自研架构的演进	3
1.2 本章小结	3
2 Framework-Defined Infrastructure：云编译器思维	4
2.1 Vercel 即”云编译器”	4
2.2 ”Vercel 不运行 Next.js”	4
2.3 基础设施原语的商品化	5
2.4 本章小结	5
3 Build vs. Buy：基础设施层的战略选择	5
3.1 为什么选择 Buy	5
3.2 Apple 硅片类比	6
3.3 Vercel 作为全球分布式元数据存储	6
3.4 本章小结	6
4 竞争策略：垂直整合与有主见的观点	6
4.1 不关注竞争对手	6
4.2 Next.js 的云原生 Trojan Horse	7
4.3 与 AWS 的共生关系	7
4.4 本章小结	7
5 消费定价与可观测性：基础设施经济学	8
5.1 消费导向的世界	8
5.2 好系统 vs. 坏系统：DynamoDB vs. PostgreSQL	8
5.3 Soft Caps 与 Hard Caps 的教训	8
5.4 实时遥测投资	9
5.5 本章小结	9
6 Compute Density：AI 时代的新型工作负载	9
6.1 AI 改变了请求模型	9
6.2 CPU-Bound vs. IO-Bound 的智能调度	9
6.3 消除客户的扩展负担	10
6.4 本章小结	10
7 v0：AI 原生软件开发的实验	10
7.1 什么是 v0	10
7.2 从 GPT-3.5 Turbo 到 Idea-Market Fit	10
7.3 All Software Will Be AI-Native	11
7.4 本章小结	11

8 创业方法论：MVP、不可变部署与 Day 1/100/1000	11
8.1 从 API 开始	11
8.2 IPFS 与 Gossip 协议的启发	12
8.3 Day 1 / Day 100 / Day 1000 框架	12
8.4 设定不合理的高标准	12
8.5 本章小结	13
9 技术人员的职业发展	13
9.1 早入行 vs. 做研究	13
9.2 招聘顶尖基础设施工程师	13
9.3 本章小结	13
10 总结与延伸	13
10.1 拓展阅读	14

1 引言：为何在 AWS 之上再建一层？

本讲是 Stanford CS 153: Infrastructure at Scale 的第五讲，嘉宾为 Vercel CEO Guillermo Rauch。讲座以 fireside chat 形式展开，围绕 Vercel 如何在 AWS 等 hyperscaler 之上构建应用层基础设施、如何应对 AI 时代的新型工作负载，以及创业过程中的技术决策与教训。

Vercel 简介

Vercel 是一家前端云平台公司，同时维护开源框架 Next.js。其核心理念是让开发者以“一条命令”完成从代码到全球部署的全流程，无需手动配置底层基础设施。Rauch 于约 2015 年创立该公司（最初名为 ZEIT），目前服务于大量企业客户，在 Black Friday 等流量高峰期间承载海量请求。

Rauch 在开场即点明 Vercel 的核心雄心：**让普通开发者能以 Squarespace/Wix 级别的简易度，构建出 Google、Amazon、Meta 级别的 Web 应用**。许多投资人曾认为这不可能实现，但云计算的新兴创新（如 serverless 函数、边缘计算等）使这一愿景逐步成为现实。

Vercel 的核心命题

Vercel 的起点不是“我有基础设施，来放应用吧”，而是反过来——“你有一个应用想法，基础设施应该是**不可见的**（invisible）”。Rauch 强调：“对开发者来说，最重要的是 idea 和 app code，基础设施必须消失在背后。”

1.1 从 Kubernetes 到自研架构的演进

Vercel 的第一个版本使用 Kubernetes 作为工作负载运行平台。然而，Rauch 很快发现 Kubernetes 过于笨重（heavy）、僵化（rigid）、静态（static），伸缩速度太慢，无法支撑“每个 git commit 即一次部署”的产品愿景。

Kubernetes 的局限

Kubernetes 虽然是容器编排的事实标准，但对于需要**极致弹性和毫秒级响应**的前端部署场景，其 Pod 调度和水平扩展的延迟往往难以接受。Rauch 直言，基于 Kubernetes 的方案“bleeding money left and right”——每个 commit 都生成一个 deployment 的模型在 Kubernetes 上经济上完全不可持续。

这段经历促使团队从零设计适合 Vercel 愿景的基础设施，最终发展出 Framework-Defined Infrastructure 的理念。

1.2 本章小结

Vercel 的创业起点是对云部署复杂性的不满。Rauch 通过将关注点从“基础设施优先”反转为“应用优先”，在 hyperscaler 之上构建了一个抽象层。第一版基于 Kubernetes 的方案因成本和弹性问题被淘汰，这一失败为后续架构创新埋下伏笔。

2 Framework-Defined Infrastructure: 云编译器思维

Rauch 将 Vercel 的方法论总结为 **Framework-Defined Infrastructure (FDI)**——框架定义基础设施。这一理念的灵感来源于 Software-Defined Networking (SDN)。

Framework-Defined Infrastructure 的核心思想

传统云服务 (AWS) 的范式是：先配置基础设施，再把应用放进去 (“Primitives over Frameworks”)。Vercel 将这一过程完全反转：开发者使用框架 (如 Next.js) 编写应用代码，部署时 Vercel 自动生成适配的基础设施。框架本身包含了 “guard rails”，赋予应用结构化的形状，基础设施成为部署的**输出**而非输入。

2.1 Vercel 即”云编译器”

Rauch 给出了一个精妙的技术类比：

“Think of your application code as the input, and think of Vercel as a cloud compiler. Vercel maintains an intermediate representation of that code being transformed into infra.”

这意味着：

1. **输入**：开发者的应用代码 (Next.js 项目)
2. **中间表示**：Vercel 对代码进行分析，识别出静态页面、动态路由、API 端点、边缘函数等组件
3. **输出**：自动编排为 S3 对象、边缘缓存、serverless 函数等基础设施原语

ISR：增量静态再生

Incremental Static Regeneration (ISR) 是 Vercel 发明的关键技术。以电商客户为例：大多数客户的后端无法承受 Black Friday 的流量洪峰。ISR 让 Vercel 按一定时间间隔调用后端，将页面”物化” (materialize) 到网络边缘，由 Vercel 吸收全部用户流量。后端压力被大幅降低，页面响应速度则接近 CDN 静态资源。

2.2 ”Vercel 不运行 Next.js”

Rauch 抛出一个看似矛盾的论断：**Vercel does not run Next.js**。这意味着 Vercel 从不简单地将 Next.js Server 扔进虚拟机然后执行 `npm start`。

”npm start” 模式的问题

如果简单地将 Next.js 应用部署到 VM 中运行：(1) multi-tenant 程度低，每个应用独占一个 VM 的时间和空间资源；(2) 无法做到全球分布式，前端用户期望 CDN 级别的延迟；(3) 无法根据工作负载特性做精细化优化。Vercel 更像是一个 CDN 而非 EC2——它将应用代码”编译”为全球分布式的基础设施原语。

Vercel 的做法是将应用拆解为高度专业化的基础设施组件：静态资源推送到边缘缓存、动态逻辑以 serverless 函数运行、数据通过多级缓存体系加速。这种分解使得前端工程师在不知不觉中获得了资深基础设施工程师级别的架构优势。

2.3 基础设施原语的商品化

Rauch 指出，hyperscaler 提供的底层原语（S3、计算实例、队列、存储等）已经高度商品化（commoditized）：

- S3 的 API 在各云服务商之间几乎完全一致
- AI 模型的调用接口也在向 OpenAI HTTP 协议标准化
- 当所有后端变成商品时，价值向应用层和前端体验聚拢

AWS "Primitives over Frameworks" 的历史合理性

AWS 早期提出“原语优先于框架”的理念在云计算初期是正确的——一切都需要从头发明。但如今基础设施已经成熟且商品化，框架层的抽象能力才是差异化竞争力所在。Vercel 的定位正是在这一商品化趋势中，做连接 hyperscaler 与开发者的桥梁。

2.4 本章小结

Framework-Defined Infrastructure 是 Vercel 的核心方法论：将应用代码视为输入，通过“云编译器”自动产出最优的基础设施配置。这一范式使前端工程师无需理解底层基础设施细节，同时享有 CDN 级别的全球分布和多级缓存优化。ISR 等技术创新是这一理念的具体实现。

3 Build vs. Buy：基础设施层的战略选择

当被问及 Vercel 是否会自建底层基础设施以替代 AWS 时，Rauch 给出了明确且深思熟虑的回答：在可预见的未来不会。

3.1 为什么选择 Buy

90% 的 IT 工作负载尚未上云

Rauch 引用了与 AWS CEO Andy Jassy 会面时的数据：全球仍有约 90% 的 IT 工作负载不在云上。这意味着将工作负载迁移到云端仍是一个巨大的增量机会。Jassy 对 Vercel 的存在“非常高兴”，因为 Vercel 降低了上云门槛，间接为 AWS 带来更多消费。

Rauch 给出不自建基础设施的理由：

1. **规模不可复制**：AWS 的闲置计算资源池“天文数字般庞大”（astronomically large），Vercel 可以在 Black Friday 按需调用这些资源
2. **商品化加速**：底层基础设施的价格持续下降，自建是净损失
3. **机会成本**：精力应投入“捕获 AI 时代的下一波软件开发浪潮”，而非建设“没人关心的数据中心”

3.2 Apple 硅片类比

Rauch 用 Apple 的芯片战略做类比：Apple 在 iPhone 已经征服世界之后，才开始设计自研芯片。消费者从不关心芯片是谁造的——他们关心的是组件的组装和最终的消费体验。Vercel 当前的机会在于做 AI 时代的桥梁，而非深入硬件层。

Build vs. Buy 的决策框架

Rauch 分享了自己在 Build vs. Buy 决策中”最大的伤疤”：Vercel 先后使用了**四种不同的数据库**来存储部署元数据。最糟糕的错误是相信自建数据库能带来业务优势——实际上，选择最适合特定场景的外部数据库、使其扩展、增加冗余层并持续迭代架构，才是正确的优先级。

过度自建的代价

Rauch 坦言自己”有过太多 DIY 的痛”。作为技术创始人，他亲自编写了 Vercel 早期的调度器、Placement 系统、负载均衡器，甚至自己的 React 和 Kubernetes 克隆版本。虽然这些经验极其宝贵，但他也承认”too much build versus buy, too much DIY can kill you”。创业者需要在技术好奇心和商业务实之间找到平衡。

3.3 Vercel 作为全球分布式元数据存储

Rauch 用一个半开玩笑的方式描述 Vercel 的本质：

“Vercel is actually a globally distributed, highly available, fast-propagating metadata store that then gets repurposed as a control plane and as a way of invoking compute.”

换言之，Vercel 的核心是一个**元数据存储与传播系统**，所有部署、路由、域名映射等信息以极快的速度在全球节点间传播。计算（serverless 函数等）是在此基础上”按需生成”的。

React for Infrastructure

Rauch 将 Vercel 比作”基础设施版的 React”：所有资源默认 spin down to zero（缩减到零），只有当用户访问时才按需创建。这种**声明式、响应式**的基础设施范式与 React 的虚拟 DOM 渲染哲学一脉相承——状态变更驱动 UI 更新，访问请求驱动基础设施存在。

3.4 本章小结

Vercel 坚定选择在 AWS 之上构建，而非自建底层基础设施。这一决策基于规模经济、商品化趋势和机会成本的综合考量。Rauch 从四次更换数据库的教训中总结出：对于非核心差异化的组件，Buy 优于 Build；公司的注意力应投向最能创造价值的方向。

4 竞争策略：垂直整合与有主见的观点

4.1 不关注竞争对手

Rauch 对竞争的第一反应是：”我非常努力地**不去想竞争**。每次我在职业生涯中沉迷于竞争分析，我都偏离了客户真正在要求的東西。”

竞争策略的核心: Opinionated Point of View

Vercel 与 Netlify、AWS Amplify、Cloudflare Pages 等竞争对手的本质区别在于: 后者都说” 去找一个适合你的框架, 我们提供基础设施”, 而 Vercel 将框架与基础设施**垂直整合**(verticalized integration)。Next.js 既是开源框架, 也是 Vercel 平台的最佳实践入口。

这种垂直整合的灵感来自两个类比:

- **Linux 发行版**: 将内核、工具链、包管理器整合为统一体验
- **Apple 硬件 + 操作系统**: 硬件与软件协同开发, 为用户提供最优整合体验

Rauch 强调, 客户喜欢 Vercel 的原因恰恰是它能” 代表客户采购最优组件” (procure the best components on their behalf), 而不是让客户自己在茫茫多的 AWS 服务中选择。

4.2 Next.js 的云原生 Trojan Horse

Rauch 将 Next.js 描述为一个” 特洛伊木马”, 在开发者使用它编写应用的同时, **悄然植入了云原生架构设计**:

Next.js 与云可移植性

Next.js 应用本身可以在任何地方运行和托管。这解决了 Docker 和 Kubernetes 最初试图解决的问题——快速上云的同时保持与特定云服务商的健康距离。但当部署到 Vercel 时, Next.js 应用会被” 编译” 为全球分布式系统, 充分利用平台的优化能力。

4.3 与 AWS 的共生关系

Rauch 与 AWS CEO Andy Jassy 的会面揭示了一个有趣的共生关系: AWS 有动力推广 Vercel 和 Next.js, 因为这降低了企业上云的门槛。传统路径要求企业先学习 CloudFormation 或 Terraform, 这正是 90% 工作负载仍未上云的原因之一。

” 免费无限” 的陷阱

当竞争对手打出” 免费无限” 的营销牌时, Rauch 直言” nothing in the world is free and unlimited”。表面上看某些平台因自有基础设施而能提供低价, 但真正的差异可能只是营销策略。基础设施的每一次流量服务都有成本, 过度承诺最终会以服务质量或隐性限制的形式反噬。

4.4 本章小结

Vercel 的竞争策略建立在” 有主见的观点” 之上: 通过框架与基础设施的垂直整合, 而非在底层原语上打价格战。这种策略使 Vercel 从众多” 提供基础设施让你自己选框架” 的竞争者中脱颖而出, 并与 AWS 形成共生而非竞争的关系。

5 消费定价与可观测性：基础设施经济学

5.1 消费导向的世界

Rauch 指出,整个基础设施世界正在向**消费定价**(consumption-based pricing)模式转型:Snowflake、Datadog、AWS 本身,以及 AI token 的采购方式都是按需消费。Vercel 也采用这一模式:”here’s what everything costs, you pay for what you use, and you pay as you grow.”

基础设施成本的反馈回路

Rauch 认为开发者需要的最佳反馈信号是:每次操作消耗了多少基础设施资源。他将此类比为土木工程师建造大楼——”用了多少材料?承重结构浇注了多少混凝土?”消费定价不是惩罚,而是将软件工程师与物理现实重新连接的反馈机制。

5.2 好系统 vs. 坏系统: DynamoDB vs. PostgreSQL

Rauch 以数据库为例阐述了好坏基础设施系统的区别:

特性	好系统 (DynamoDB/CosmosDB)	坏系统 (某些关系型 DB)
成本反馈	每个操作返回精确的消耗	整体性能下降,无法归因
扩展方式	水平自动扩展	需手动调优, noisy neighbor
开发者感知	知道查询花费 1.23 分	不知道哪个 join 拖垮了系统
服务质量	付费换取一致的 QoS	昂贵查询影响所有工作负载

表 1: 基础设施系统的成本反馈对比

Noisy Neighbor 问题

在共享基础设施中,某个客户的高负载查询可能导致其他客户的服务质量下降。Rauch 特别提到他对 noisy neighbor 问题的”执念”:”\$4 的 VPS 很酷,但流量一上来,你的客户就会被其他租户拖累。”Vercel 必须精细地分配 CPU 和内存资源,确保性能不因个别客户的流量激增而退化。

5.3 Soft Caps 与 Hard Caps 的教训

Vercel 引入了软硬上限机制来管理消费:

- **Soft Cap**: 提供预警和学习信号,告知用户接近预算上限
- **Hard Cap**: 绝对不超过的消费上限

Hard Cap 的双刃剑

Rauch 分享了一个真实案例:某客户因 Hard Cap 生效而**损失了 \$20,000 的收入**——当时正值流量高峰,客户其实希望基础设施继续运行以服务付费用户。OpenAI 后来也在其 hard cap 功能上添加了警告:”设置 hard cap 可能对你的业务不利。”教训是:消费上限必须作为**收入或增长的函数**来考虑,而非绝对金额。

5.4 实时遥测投资

Vercel 在可观测性上投入巨大，构建了世界上最实时的使用量监控系统之一：

- 分钟级的基础设施账单更新（而非竞品的每日/每 6 小时汇总）
- 企业用户支付的 platform fee 中很大一部分用于支撑这套实时遥测基础设施
- 目标是让开发者拥有与土木工程师相同的”物理世界感知”——写的每一行代码都有明确的成本信号

基础设施的透明度原则

Rauch 的定价哲学核心是**透明与清晰**（transparency and clarity）。一切都有成本，最好的策略是让用户清楚地知道成本在哪里，而非用”免费无限”的营销话术掩盖真实经济学。

5.5 本章小结

Vercel 采用消费定价模式，核心是为开发者构建精确的成本反馈回路。好的基础设施系统（如 DynamoDB）会告诉你每个操作的成本，使开发者能做出知情的优化决策。Soft/Hard Cap 机制需要谨慎设计以避免业务损失。实时遥测系统是 Vercel 差异化投资的重要方向。

6 Compute Density: AI 时代的新型工作负载

6.1 AI 改变了请求模型

Rauch 宣布 Vercel 正在推出一款针对 AI 工作负载优化的新计算产品（于 2025 年 2 月 4 日正式发布）。其背景是 AI 从根本上改变了 Web 请求的特征：

从毫秒到分钟：请求模型的根本变革

传统 Web 应用追求**瞬时响应**：电商交易、OLTP 数据库查询的 P99 延迟在 10 毫秒级别。但 AI 推理（尤其是 reasoning model 如 OpenAI o1、DeepSeek R1 等）可能让一个请求持续**数分钟**。这意味着计算资源需要为单个请求保留更长时间，彻底颠覆了传统 serverless 的假设。

6.2 CPU-Bound vs. IO-Bound 的智能调度

Vercel 新计算产品的核心创新是 **Compute Density**——根据工作负载特性智能调整资源分配：

工作负载类型	特征	Vercel 策略
CPU-Bound	计算密集，需要 CPU 周期	激进水平扩展，为每个请求保留 CPU
IO-Bound	等待外部服务（如 AI 推理）	智能饱和已分配计算资源

表 2: Compute Density 的工作负载自适应策略

Servers meet Serverless

Rauch 用“servers meet serverless”来描述这一新范式。传统 serverless 的缺陷是对长时间运行的请求不友好（冷启动、超时限制等），而传统服务器（VPS/EC2）的缺陷是需要用户手动管理扩展。Vercel 的 Compute Density 方案试图结合两者优势：像 serverless 一样免运维，像 server 一样高效利用资源。

6.3 消除客户的扩展负担

Rauch 强调，Vercel 的目标是**零配置扩展**——彻底消除以下传统负担：

- 手动调整 Kubernetes Pod 数量
- 不断调优 CPU 阈值
- 决定需要多少 VPS 实例
- 权衡 auto-scaling 参数

将扩展负担推给客户的危害

Rauch 指出，最糟糕的云平台设计是“将可扩展性的负担放在客户身上”。手动调优 Pod 数量、CPU 阈值等参数不仅耗费工程师时间（TCO），还容易导致配置不当引发的故障。理想的平台应当根据工作负载画像自动做出最优决策。

6.4 本章小结

AI 时代的 Web 请求模型从“毫秒级响应”扩展到“分钟级等待”，对基础设施提出了全新要求。Vercel 的 Compute Density 产品通过区分 CPU-Bound 和 IO-Bound 工作负载，实现智能资源分配，在保持服务质量的同时降低客户成本。“Servers meet Serverless”代表了云计算演进的下一步。

7 v0：AI 原生软件开发的实验

7.1 什么是 v0

v0 (v0.dev) 是 Vercel 推出的 AI Web 开发助手。用户输入自然语言 prompt，v0 将其转换为可工作的前端/全栈应用，使用 React、Next.js、Tailwind CSS、shadcn/ui 等 Vercel 生态工具链。

v0 的爆炸式增长

v0 是 Vercel 增长最快的产品——在 14-20 天内收入翻了三番。它打开了漏斗的顶部 (top of funnel)，让任何有想法的人——不仅仅是专业开发者——都能创建软件。Rauch 的一位同事甚至用 v0 重建了整个 Vercel Dashboard。

7.2 从 GPT-3.5 Turbo 到 Idea-Market Fit

v0 的第一个版本建立在 GPT-3.5 Turbo 之上，当时甚至**没有 function calling** 功能。Vercel 团队不得不在模型之上自建了一套 bespoke 函数调用系统。

Idea-Market Fit vs. Product-Market Fit

Rauch 区分了两个概念：v0 一开始就有 **Idea-Market Fit**——AI 生成代码的想法本身就有巨大需求。但要达到 **Product-Market Fit**，团队需要不断推进模型能力和工程创新的边界。这个从“想法验证”到“产品成熟”的过程是持续的研发投入。

7.3 All Software Will Be AI-Native

Rauch 表达了一个强烈的观点：**所有软件最终都将是 AI 原生的**。这不仅指用 AI 辅助写代码，还包括：

- 过去需要设计大量 UI 和表单的场景，agent 可以自动化
- 编写代码、构建、部署的全流程可以通过 AI 对话界面抽象
- v0 本身就是这一愿景的产品化落地

在公众面前做研究的痛苦

Rauch 坦言“researching in public with your customers is more painful but can be quite satisfying”。v0 的早期版本能力有限，但持续迭代和公开交付让团队获得了宝贵的用户反馈。这种“公开研发”模式对于创业公司来说既是风险也是加速器。

7.4 本章小结

v0 代表了 Vercel 对 AI 原生软件开发的押注。从 GPT-3.5 时代的简陋原型到快速增长的明星产品，v0 的发展历程验证了“idea-market fit 先行，product-market fit 随后”的创业方法论。Rauch 坚信所有软件都将走向 AI 原生，这既是 Vercel 的产品方向，也是其基础设施需求的核心驱动力。

8 创业方法论：MVP、不可变部署与 Day 1/100/1000

8.1 从 API 开始

Rauch 分享了他对 MVP 的独到见解。他在天使投资中观察到，**最好的公司往往从一两个 API 调用开始**。他投资的三家成功公司都遵循这一模式：

公司	起步形态
Auth0	一个认证 API
Clearbit	一个人物信息查询 API
Scale AI	一个人工标注数据 API

表 3: 三家从单一 API 起步的成功公司

受此启发，Vercel 的第一版产品就是一个**部署 API**——输入代码，返回一个不可变的 URL。

不可变部署的哲学

Rauch 深受函数式编程和 React 的 immutability 理念影响。他将部署设计为**不可变的** (immutable), 就像 git commit 不可变一样——每次部署生成一个唯一的、永不改变的 URL。这个设计决策看似简单, 实际上对底层基础设施提出了极高要求 (需要高效的内容寻址存储和全球传播系统)。

8.2 IPFS 与 Gossip 协议的启发

Rauch 透露, Vercel 的架构灵感部分来自去中心化系统 IPFS:

Content-Addressable Store 与 Gossip Protocol

IPFS 是一个全球内容寻址存储系统——通过 checksum 即可获取任何内容, 甚至不需要知道网络的完整成员信息。Vercel 的底层基础设施大量使用了类似的 gossip protocol, 例如: 安全系统需要实时分析流量画像并阻断攻击者, 部署信息 (如 stanford.edu 应指向哪个不可变部署) 需要在全球数百万台计算机之间快速传播。

8.3 Day 1 / Day 100 / Day 1000 框架

Rauch 分享了他最重要的产品方法论之一:

Day 1 / Day 100 / Day 1000 评估框架

- **Day 1:** 首次使用体验。”一条命令部署”可以吸引大量早期用户
- **Day 100:** 持续使用体验。随着项目代码增长、功能增加, 平台是否”泄漏” (leak)? 能否持续吸收复杂性?
- **Day 1000:** 运维体验。系统是否稳定在线? Vercel 在 Black Friday 期间达到了 **9 个 9** 的可用性 (99.9999999%)

在基础设施领域, **没有人真正关心 Day 1 体验**——Day 100 和 Day 1000 才是决定胜负的关键。

漂亮 Demo 的陷阱

很多基础设施产品可以做出惊艳的 Day 1 demo, 但在 Day 100 就开始”泄漏”——性能退化、抽象失效、隐藏的复杂性暴露。Rauch 强调, Vercel 创立后的**整整一年多**都在追赶 Day 100 和 Day 1000 的质量标准, 而非打磨 Day 1 的花哨体验。

8.4 设定不合理的高标准

Rauch 的个人哲学是**设定不合理的高标准** (unrealistic expectations), 然后让标准本身成为研究项目。例如:

- ”每个 git commit 必须成为一次部署”——这在 Kubernetes 上不可行, 迫使团队发明全新的基础设施

- v0 在 GPT-3.5 时代就开始构建——当时模型能力远不足以实现愿景,但推动了团队在 function calling 等方面的创新

8.5 本章小结

Vercel 的创业路径遵循”从最小 API 开始验证 idea-market fit, 然后用一年以上的的时间追赶 Day 100/Day 1000 的质量标准”。不可变部署的设计理念来自函数式编程和去中心化系统的启发, 而 gossip protocol 等技术被实际应用于 Vercel 的全球传播系统。设定不合理的高标准既是痛苦的来源, 也是创新的驱动力。

9 技术人员的职业发展

9.1 早人行 vs. 做研究

当被问及是否后悔 18 岁就进入工业界（全职前端工程师）而非继续学术研究时, Rauch 表示毫无遗憾。他的理由是：

1. 他加入的公司本身就给了他做研究的机会——创业公司的技术挑战本身就是研究项目
2. 通过实践学习构建系统的速度”非凡” (phenomenal)
3. 可以设定不合理的产品目标, 让”达标”的过程本身成为新颖的研究

以产品为驱动的研究

Rauch 提出了一种”以产品驱动研究”的模式：不是先做研究再产品化, 而是设定一个激进的产品目标（如”每个 commit 即部署”), 然后在实现过程中自然产生新颖的研究问题。v0 也是这一模式的典型案例——先有 idea-market fit, 再通过推动 state-of-the-art 来达到 product-market fit。

9.2 招聘顶尖基础设施工程师

Rauch 提到 Vercel 基础设施团队的招聘极其困难, 因为”没有人见过我们所处的规模”。团队成员通常在公司待很长时间, 因为他们面对的是”互联网上最迷人的挑战”。Rauch 强调他亲自 out-bound 联系领域内最优秀的人才。

9.3 本章小结

对于技术人员的职业发展, Rauch 倡导通过实践而非纯学术路径来积累能力。创业公司的极端技术挑战本身就是最好的研究训练场。关键在于设定足够高的标准, 让达标过程自然产生创新。

10 总结与延伸

本讲中, Guillermo Rauch 从多个维度分享了 Vercel 在 Web 基础设施领域的思考与实践：

1. **Framework-Defined Infrastructure**: 应用代码是输入, Vercel 作为”云编译器”自动产出最优基础设施

2. **Build vs. Buy**: 在 hyperscaler 之上构建抽象层, 而非自建底层, 将精力聚焦于最能创造价值的应用层
3. **消费定价与可观测性**: 精确的成本反馈回路使开发者能做出知情决策, soft/hard cap 需要作为业务函数考虑
4. **Compute Density**: AI 时代的请求模型 (毫秒到分钟) 要求全新的计算调度策略
5. **v0 与 AI 原生**: 所有软件最终都将是 AI 原生的, v0 是这一愿景的产品化落地
6. **Day 1/100/1000**: 基础设施产品的成功取决于长期可靠性而非首次体验的花哨

核心启示

Rauch 的思维模型贯穿一个主题: **从用户体验出发反推基础设施需求**。无论是”基础设施应当不可见”的产品哲学、”框架定义基础设施”的技术范式, 还是”Day 1000 才是真正的考验”的运维标准, 都体现了以终端用户为中心的工程思维。在 AI 时代, 这一思维更加重要——当请求模型从毫秒扩展到分钟, 平台必须在无需用户干预的情况下自适应。

10.1 拓展阅读

- **Vercel 官方文档**: <https://vercel.com/docs> —涵盖 Framework-Defined Infrastructure、ISR 等核心技术的详细说明
- **Next.js 文档**: <https://nextjs.org/docs> —了解框架层如何与 Vercel 平台协同
- **v0 产品**: <https://v0.dev> —AI 驱动的 Web 开发体验
- **Vercel AI SDK**: <https://sdk.vercel.ai> —Vercel 的 AI 集成工具包, 支持多模型 streaming 等能力
- **Software-Defined Networking (SDN)**: 了解 FDI 理念的灵感来源
- **IPFS 白皮书**: <https://ipfs.tech> —Content-addressable store 和 gossip protocol 的详细设计
- **Amazon DynamoDB 定价模型**: 作为消费定价和成本反馈回路的典型案例